

SUPPORT DE COURS COMPLET

Cryptanalyse

16h CM · 8h TP · 24h

Dr. Zakaria Sawadogo

École Polytechnique de Ouagadougou



Sommaire

Syllabus

1. Chapitre 1 — Introduction à la cryptanalyse : histoire et classification des attaques
2. Chapitre 2 — Cryptanalyse des chiffrements classiques
3. Chapitre 3 — Cryptanalyse des chiffrements symétriques modernes
4. Chapitre 4 — Attaques sur les fonctions de hachage
5. Chapitre 5 — Cryptanalyse des systèmes asymétriques
6. Chapitre 6 — Attaques par canaux auxiliaires et attaques pratiques

Cryptanalyse

Volume horaire : 16h CM · 8h TP (24h)

Objectifs pédagogiques

- Comprendre les principes et l'histoire de la cryptanalyse, de la substitution simple aux attaques sur les systèmes modernes.
- Analyser les faiblesses des chiffrements classiques par analyse fréquentielle.
- Connaître les grandes familles d'attaques contre les chiffrements symétriques, les fonctions de hachage et les systèmes asymétriques.
- Comprendre les attaques par canaux auxiliaires et leur implication pratique en sécurité des systèmes.

Prérequis

Le cours *Cryptographie* pose les bases nécessaires (chiffrement symétrique/asymétrique, fonctions de hachage) ; il est recommandé de suivre les deux modules en parallèle ou de suivre *Cryptographie* en premier. Des notions de probabilités élémentaires et d'arithmétique modulaire sont utiles.

Plan de séances

Cours magistraux (16h)

Séance	Chapitre	Durée
1	Introduction à la cryptanalyse : histoire et classification des attaques	2h
2	Cryptanalyse des chiffrements classiques	3h
3	Cryptanalyse des chiffrements symétriques modernes	3h
4	Attaques sur les fonctions de hachage	2h
5	Cryptanalyse des systèmes asymétriques	3h
6	Attaques par canaux auxiliaires	3h

Travaux pratiques (8h)

Séance	Sujet	Durée
1	Cryptanalyse par analyse fréquentielle	2h

Séance	Sujet	Durée
2	Attaques par force brute et dictionnaire sur hachages	2h
3	Attaque sur le mode ECB et padding oracle	2h
4	Cryptanalyse de RSA mal implémenté	2h

Évaluation

- TP notés : 40 %
- Exposé sur une attaque cryptographique historique ou récente (au choix) : 20 %
- Examen final : 40 %

Avertissement éthique

Les techniques enseignées dans ce module s'exercent exclusivement sur des exemples pédagogiques (messages, systèmes ou clés fournis pour l'exercice). Leur usage contre un système réel sans autorisation explicite est illégal et hors du cadre de ce cours.

Bibliographie

- D. Kahn, *The Codebreakers*.
- A. Menezes, P. van Oorschot, S. Vanstone, *Handbook of Applied Cryptography* (disponible librement en ligne).
- J. Katz, Y. Lindell, *Introduction to Modern Cryptography*.

Chapitre 1 — Introduction à la cryptanalyse : histoire et classification des attaques

1. Définition

La cryptanalyse est l'étude des méthodes permettant de retrouver, en tout ou partie, l'information protégée par un système cryptographique (message clair, clé) sans disposer légitimement de la clé de déchiffrement. Elle est le pendant offensif de la cryptographie, et les deux disciplines progressent ensemble : un algorithme n'est considéré fiable qu'après avoir résisté longtemps à un examen cryptanalytique public et intensif (principe de Kerckhoffs, vu au module *Cryptographie*).

Il est important de distinguer plusieurs objectifs possibles d'une cryptanalyse, du plus fort au plus faible :

- **Rupture totale (total break)** : retrouver la clé secrète elle-même. C'est l'objectif le plus ambitieux, mais aussi le plus rarement atteint contre un algorithme moderne bien conçu.
- **Déduction globale** : retrouver un algorithme fonctionnellement équivalent à l'algorithme légitime, sans connaître la clé exacte.
- **Déduction locale (ou déduction d'information)** : retrouver de l'information partielle sur un clair ou une clé — par exemple quelques bits de clé, ou la capacité de distinguer un chiffré d'une suite aléatoire (attaque par distinction). C'est le type de résultat le plus fréquent en recherche académique : une cryptanalyse « théorique » qui réduit la sécurité d'un algorithme de 2^n à $2^{(n-10)}$, par exemple, ne casse pas le système en pratique mais constitue un signal d'alerte scientifique important.
- **Distinction (distinguishing attack)** : montrer qu'il existe un moyen de différencier la sortie de l'algorithme d'une sortie parfaitement aléatoire, sans nécessairement retrouver la clé. C'est souvent la première étape qui ouvre la voie à des attaques plus poussées.

Cette gradation explique pourquoi on lit parfois qu'un algorithme est « cassé » alors qu'il reste, en pratique, totalement sûr pour un usage réel : le terme recouvre des niveaux de gravité très différents, et seule une lecture précise du résultat (complexité, modèle d'attaque, objectif atteint) permet d'en évaluer l'impact réel.

2. Repères historiques

- **Antiquité à Renaissance** : chiffrement de César, chiffres de substitution ; cassés par analyse fréquentielle dès le IX^e siècle par le savant arabe Al-Kindi, dans son traité *Sur le déchiffrement des messages cryptographiques*, considéré comme le premier texte connu de cryptanalyse systématique.
- **XVI^e siècle** : chiffre de Vigenère, longtemps considéré incassable (« le chiffre indéchiffrable »), cassé au XIX^e siècle par Kasiski et Babbage.
- **1917, télégramme Zimmermann** : un télégramme diplomatique allemand chiffré, intercepté et déchiffré par les cryptanalystes britanniques de la « Room 40 », révèle une proposition d'alliance

avec le Mexique contre les États-Unis ; sa divulgation contribue à l'entrée en guerre des États-Unis. Cet épisode illustre très tôt l'impact géopolitique direct que peut avoir une cryptanalyse réussie.

- **Seconde Guerre mondiale** : cryptanalyse de la machine Enigma. Les travaux précurseurs du bureau du chiffre polonais (Marian Rejewski, Jerzy Różycki, Henryk Zygalski), qui exploitent dès les années 1930 des faiblesses de procédure et développent la « bombe » électromécanique polonaise, sont transmis aux Alliés en 1939 et poursuivis à Bletchley Park par les équipes britanniques (Alan Turing, Gordon Welchman notamment), qui perfectionnent la Bombe de Turing pour automatiser la recherche de réglages. Ce travail est considéré comme un tournant fondateur à la fois pour la cryptanalyse moderne et pour la naissance de l'informatique (Turing y développe des concepts qui nourriront sa réflexion sur le calcul).
- **Ère moderne** : cryptanalyse différentielle et linéaire contre les chiffrements par blocs (DES), attaques mathématiques contre RSA (factorisation), attaques par canaux auxiliaires exploitant l'implémentation physique plutôt que l'algorithme.

Zoom : pourquoi Enigma était-elle vulnérable malgré un espace de clés énorme ?

Enigma combinait des rotors interchangeable, un réflecteur et un tableau de connexions (*plugboard*), offrant un espace de réglages théoriquement gigantesque. Le système n'a pourtant pas été cassé par force brute, mais par l'exploitation de **faiblesses structurelles et procédurales** :

- le réflecteur garantissait qu'aucune lettre ne pouvait être chiffrée en elle-même — une contrainte qui, combinée à des mots probables (*cribs*, comme des formules météo répétitives ou des salutations standardisées), réduisait drastiquement l'espace de recherche ;
- des habitudes opérationnelles prévisibles des opérateurs (réglages réutilisés, messages stéréotypés) fournissaient des points d'appui réguliers aux cryptanalystes.

Cet exemple illustre un principe qui traverse tout ce cours : **la robustesse théorique d'un système ne suffit pas si sa mise en œuvre introduit des régularités exploitables.**

3. Classification des attaques selon les informations disponibles

Type d'attaque	Information disponible à l'attaquant
Ciphertext-only (texte chiffré seul)	uniquement des messages chiffrés
Known-plaintext (texte clair connu)	un ou plusieurs couples (clair, chiffré)
Chosen-plaintext (texte clair choisi)	l'attaquant peut faire chiffrer des textes de son choix
Chosen-ciphertext (texte chiffré choisi)	l'attaquant peut faire déchiffrer des textes chiffrés de son choix (hors le message cible)

Plus l'attaquant dispose de capacités (de ciphertext-only à chosen-ciphertext), plus le modèle est puissant, et plus un algorithme résistant à un modèle fort est considéré robuste. On distingue aussi des variantes utiles en pratique :

- **Adaptive chosen-plaintext/ciphertext** : l'attaquant choisit ses requêtes successives en fonction des réponses déjà obtenues, ce qui est souvent bien plus puissant qu'un ensemble de requêtes fixé à l'avance (c'est le modèle réaliste des attaques par *oracle*, comme l'attaque par *padding oracle* étudiée au chapitre 6).
- **Related-key attack** : l'attaquant obtient des chiffrés produits sous plusieurs clés mathématiquement liées entre elles (par exemple différant d'un XOR connu). Ce modèle, moins réaliste dans l'absolu, s'avère pertinent contre certains protocoles où la dérivation de clé est mal conçue.

En pratique, le modèle *chosen-plaintext* (CPA) est devenu le standard minimal exigé pour un chiffrement moderne (sécurité IND-CPA), et le modèle *chosen-ciphertext* (CCA, en particulier CCA2 adaptatif) est requis pour les schémas de chiffrement à clé publique utilisés en pratique.

4. Classification des attaques selon la méthode

- **Attaques statistiques** : exploitation des propriétés statistiques du texte clair (fréquence des lettres, des mots) ou du chiffrement.
- **Attaques par force brute (exhaustive search)** : test systématique de toutes les clés possibles. Sa complexité croît exponentiellement avec la taille de la clé, ce qui en fait la référence de comparaison : toute attaque « intéressante » d'un point de vue cryptanalytique doit être **plus rapide** que la force brute pour être considérée comme une avancée.
- **Attaques structurelles/mathématiques** : exploitation d'une faiblesse mathématique de l'algorithme (factorisation, logarithme discret, biais différentiels ou linéaires).
- **Attaques par canaux auxiliaires (side-channel)** : exploitation d'informations physiques fuitées par l'implémentation (temps d'exécution, consommation électrique) plutôt que de l'algorithme lui-même.
- **Attaques par ingénierie sociale ou implémentation** : exploitation d'erreurs d'implémentation (générateur aléatoire faible, réutilisation de clé, mode opératoire mal choisi) plutôt que de l'algorithme mathématique — en pratique, la source la plus fréquente de compromission de systèmes cryptographiques réels.

Complexité : quelques ordres de grandeur pour situer une attaque

Un réflexe essentiel en cryptanalyse est de savoir situer un nombre d'opérations sur une échelle réaliste. Le tableau suivant (purement illustratif, à but pédagogique) aide à interpréter des complexités exprimées en 2^n :

Complexité	Ordre de grandeur	Interprétation qualitative
$2^{20} \approx 10^6$	environ un million	trivial, instantané sur un ordinateur personnel
$2^{40} \approx 10^{12}$	mille milliards	rapide sur une machine actuelle

Complexité	Ordre de grandeur	Interprétation qualitative
$2^{56} \approx 7 \times 10^{16}$	espace de clé de DES	atteignable avec du matériel dédié (illustre pourquoi DES est obsolète)
2^{80}	—	jugé insuffisant depuis longtemps comme marge de sécurité à long terme
2^{128}	espace de clé d'AES-128	hors de portée de toute force brute envisageable avec la technologie actuelle et prévisible

Ce tableau ne donne aucune estimation de temps ou de coût matériel précis (qui dépendent de paramètres technologiques évolutifs), mais illustre la **croissance exponentielle** qui fonde la sécurité par force brute : chaque bit de clé supplémentaire double le travail de l'attaquant.

5. Notion de sécurité et de coût d'une attaque

Un système est dit sûr non pas parce qu'il est mathématiquement incassable dans l'absolu, mais parce que le coût de la meilleure attaque connue (temps de calcul, mémoire, nombre de messages requis) dépasse très largement ce qu'un attaquant réaliste peut mobiliser. La sécurité est donc **relative et évolutive** : un algorithme sûr aujourd'hui peut devenir vulnérable avec de nouvelles techniques d'attaque ou une puissance de calcul accrue (ex. menace future de l'informatique quantique sur RSA, abordée au chapitre 5).

Sécurité inconditionnelle contre sécurité calculatoire

Claude Shannon a formalisé en 1949 la notion de **sécurité parfaite (inconditionnelle)** : un système offre une sécurité parfaite si le texte chiffré ne révèle strictement aucune information sur le clair, quelle que soit la puissance de calcul de l'attaquant (c'est le cas du masque jetable — *one-time pad* — utilisé correctement). En pratique, cette sécurité inconditionnelle est rarement utilisable, car elle exige une clé aussi longue que le message, utilisée une seule fois, ce qui pose des problèmes de distribution insurmontables à grande échelle.

La quasi-totalité des systèmes cryptographiques modernes (AES, RSA, ECC) reposent donc sur une **sécurité calculatoire** : ils ne sont pas prouvés incassables, mais on estime que les casser demande des ressources de calcul irréalistes avec les meilleurs algorithmes connus à ce jour. C'est une distinction fondamentale à garder en tête tout au long de ce cours : la cryptanalyse ne cherche pas à démontrer l'impossibilité d'une attaque, mais à faire progresser (ou à borner) le coût de la meilleure attaque connue.

6. Méthodologie générale d'une cryptanalyse

1. Comprendre précisément l'algorithme et son mode d'utilisation (mode opératoire, gestion des clés).
2. Identifier le modèle d'attaque réaliste (quelles informations l'attaquant peut-il obtenir ?).
3. Rechercher des biais statistiques, structurels ou d'implémentation exploitables.

4. Évaluer le coût de l'attaque (complexité temporelle, données nécessaires) et le comparer à un budget attaquant réaliste.
5. Le cas échéant, implémenter une preuve de concept sur un exemple contrôlé.

Illustration pédagogique : une attaque exhaustive triviale en Python

L'exemple suivant, volontairement simplifié, illustre les étapes 1, 2 et 5 de la méthodologie sur le chiffre de César (espace de clés minuscule : 26 valeurs). Il ne s'agit évidemment pas d'un algorithme réaliste, mais d'un support pour raisonner sur le **coût** d'une attaque exhaustive avant de l'appliquer, en apprentissage, à des espaces plus grands (chapitre 3).

```
# Attaque exhaustive pédagogique contre le chiffre de César
# (à but strictement didactique : espace de clés = 26)

ALPHABET = "abcdefghijklmnopqrstuvwxyz"

def dechiffrer_cesar(texte_chiffre, decalage):
    resultat = []
    for c in texte_chiffre:
        if c.lower() in ALPHABET:
            idx = (ALPHABET.index(c.lower()) - decalage) % 26
            resultat.append(ALPHABET[idx])
        else:
            resultat.append(c)
    return "".join(resultat)

def attaque_exhaustive(texte_chiffre):
    """Étape 5 de la méthodologie : preuve de concept.
    On énumère les 26 clés possibles (étape 2 : modèle ciphertext-only,
    espace de clé trivial) et on affiche chaque hypothèse pour
    évaluation humaine (étape 4 : coût quasi nul ici)."""
    for cle in range(26):
        print(f"clé={cle:2d} -> {dechiffrer_cesar(texte_chiffre, cle)}")

# Exemple fictif : "khor" est le chiffré de "hello" avec un décalage de 3
attaque_exhaustive("khor")
```

Ce programme énumère les 26 hypothèses de clé, à charge pour l'analyste (ou un score de vraisemblance linguistique automatisé, voir chapitre 2) de reconnaître la bonne hypothèse. Le point pédagogique central est que **le coût de l'étape 5 dépend directement de la taille de l'espace de clé identifié à l'étape 2** : ici, 26 essais ; contre AES-128, 2^{128} essais, ce qui change radicalement la faisabilité pratique de l'approche.

À retenir

- La cryptanalyse et la cryptographie évoluent en miroir : un algorithme n'est validé que par une résistance prouvée à la cryptanalyse publique.
- Le modèle d'attaque (ciphertext-only à chosen-ciphertext, avec ou sans adaptativité) détermine la puissance réaliste de l'attaquant, et donc la portée réelle d'un résultat de cryptanalyse.

- Une cryptanalyse peut viser des objectifs de gravité très différente (distinction, déduction partielle, rupture totale) : un algorithme « cassé » académiquement n'est pas nécessairement dangereux en pratique.
- La sécurité cryptographique moderne est presque toujours **calculatoire**, non **inconditionnelle** : elle repose sur le coût prohibitif de la meilleure attaque connue, une notion relative et évolutive dans le temps.
- L'histoire (Enigma en particulier) montre qu'un espace de clés immense ne protège pas contre des faiblesses structurelles ou des habitudes opérationnelles prévisibles.
- En pratique, la majorité des compromissions viennent d'erreurs d'implémentation, pas de failles mathématiques dans l'algorithme lui-même.

Chapitre 2 — Cryptanalyse des chiffrements classiques

1. Rappel : chiffrement de César et substitution mono-alphabétique

Le chiffre de César décale chaque lettre d'un nombre fixe de positions (clé de 0 à 25). Une substitution mono-alphabétique généralise le principe en remplaçant chaque lettre par une autre selon une permutation quelconque de l'alphabet.

Il est instructif de comparer les deux espaces de clés : le chiffre de César n'offre que 26 clés possibles, tandis qu'une substitution mono-alphabétique générale sur un alphabet de 26 lettres en offre 26! (factorielle de 26), soit environ 4×10^{26} permutations — un nombre bien supérieur à l'espace de clé d'AES-128 ($2^{128} \approx 3,4 \times 10^{38}$, certes plus grand, mais du même ordre de grandeur de démesure apparente). Ce constat est pédagogiquement important : **un espace de clé gigantesque ne garantit en rien la sécurité d'un algorithme** si sa structure laisse filtrer de l'information exploitable — ici, la distribution de fréquence des lettres, totalement indépendante de la taille de l'espace de clé.

2. Analyse fréquentielle

Chaque langue possède une distribution caractéristique de fréquence des lettres (en français : E $\approx$ 14,7 %, A $\approx$ 7,6 %, S $\approx$ 7,9 %, etc.). Une substitution mono-alphabétique conserve cette distribution en la « déplaçant » : la lettre la plus fréquente du texte chiffré correspond très probablement à la lettre la plus fréquente de la langue.

Méthode

1. Compter la fréquence de chaque lettre dans le texte chiffré.
2. Comparer au profil de fréquence connu de la langue.
3. Émettre des hypothèses de correspondance pour les lettres les plus fréquentes.
4. Affiner par l'analyse des digrammes/trigrammes fréquents (« ES », « LE », « ENT » en français) et par le sens du texte partiellement déchiffré.
5. Itérer jusqu'à obtenir un texte cohérent.

Pour le chiffre de César spécifiquement, il suffit de tester les 25 décalages possibles (attaque exhaustive triviale, espace de clé minuscule) ; l'analyse fréquentielle n'est même pas strictement nécessaire, mais elle permet d'automatiser le choix du bon décalage sans intervention humaine, ce qui devient précieux dès que le volume de texte à traiter augmente.

Illustration : automatiser le choix du décalage par score fréquentiel

Le programme suivant illustre comment transformer l'étape « émettre des hypothèses » en un score numérique, en comparant la distribution du texte déchiffré candidat à la distribution théorique du français (valeurs arrondies, à but strictement illustratif).

```
# Score fréquentiel pédagogique pour départager les hypothèses de décalage

FREQ_FR = { # fréquences approximatives en français (%), à but illustratif
    'a': 7.6, 'e': 14.7, 'i': 7.5, 's': 7.9, 't': 7.2, 'n': 7.1, 'r': 6.6,
    'u': 6.3, 'l': 5.5, 'o': 5.3, 'd': 3.7, 'c': 3.3, 'p': 3.0, 'm': 3.0,
}

def score_frequentiel(texte):
    """Somme des fréquences théoriques des lettres présentes : un texte
    en français plausible obtient un score élevé, un charabia un score
    plus faible (heuristique simple, pas une preuve)."""
    return sum(FREQ_FR.get(c, 0) for c in texte.lower())

def meilleure_hypothese(texte_chiffre, dechiffrer_cesar):
    scores = []
    for cle in range(26):
        clair_candidat = dechiffrer_cesar(texte_chiffre, cle)
        scores.append((score_frequentiel(clair_candidat), cle, clair_candidat))
    scores.sort(reverse=True)
    return scores[0] # meilleure hypothèse selon le score fréquentiel
```

Ce type de score, combiné à un dictionnaire de mots valides, est la base des outils automatiques de cassage de substitutions simples : plutôt que de tester manuellement 26 hypothèses, on laisse la machine classer les candidats par plausibilité linguistique.

3. Le chiffre de Vigenère et son cassage

Le chiffre de Vigenère applique un décalage de César différent à chaque position, répété selon une clé de longueur L . Il résiste à l'analyse fréquentielle directe (la distribution des lettres chiffrées est « aplatie »), mais reste cassable :

Étape 1 — Trouver la longueur de la clé

- **Méthode de Kasiski** : repérer les répétitions de séquences de lettres identiques dans le texte chiffré ; la distance entre deux occurrences est probablement un multiple de la longueur de clé. Le PGCD des distances observées donne une estimation de L . Cette méthode fonctionne d'autant mieux que le texte est long : plus le texte clair contient de répétitions naturelles (mots courants, terminaisons), plus les coïncidences dans le texte chiffré, alignées sur un multiple de L , sont nombreuses.
- **Indice de coïncidence** : mesure statistique de la probabilité que deux lettres tirées au hasard dans un texte soient identiques. Un texte en langue naturelle a un indice de coïncidence élevé ($\approx 0,0778$ en français) ; un texte purement aléatoire a un indice bas ($\approx 1/26 \approx 0,0385$). En découpant le texte chiffré en L sous-suites (une par position de clé) et en testant plusieurs valeurs de L , on retient la valeur pour laquelle l'indice de coïncidence moyen des sous-suites se rapproche de celui de la langue.

L'indice de coïncidence d'un texte de longueur N , où n_i désigne le nombre d'occurrences de la i -ème lettre de l'alphabet, se calcule par :

$$IC = \sum [n_i \times (n_i - 1)] / [N \times (N - 1)]$$

Intuitivement, ce calcul estime la probabilité qu'en tirant deux lettres au hasard sans remise dans le texte, elles soient identiques. Un texte chiffré par simple décalage (comme chaque sous-suite d'un Vigenère correctement découpé) conserve exactement la même distribution que le texte clair sous-jacent, à une permutation circulaire près — d'où un IC proche de celui de la langue naturelle dès que L est correctement deviné, et un IC proche de l'aléatoire uniforme sinon (les lettres de différentes sous-clés se mélangent).

Étape 2 — Casser chaque sous-alphabet

Une fois L connue, chaque sous-suite est un simple chiffre de César indépendant, cassable par analyse fréquentielle classique (section précédente). Une variante plus robuste consiste, pour chaque sous-suite, à calculer une corrélation croisée entre la distribution observée et la distribution théorique décalée de chaque valeur possible (0 à 25), et à retenir le décalage donnant la meilleure corrélation — c'est l'équivalent statistique, généralisé, du score fréquentiel présenté en section 2.

Exemple illustratif simplifié (chiffres fictifs, à but pédagogique)

Supposons un texte chiffré dans lequel la séquence de quatre lettres `xkcd` réapparaît trois fois, aux positions 10, 34 et 58 (exemple purement fictif construit pour l'illustration). Les distances entre occurrences sont $34 - 10 = 24$ et $58 - 34 = 24$; le PGCD de ces distances vaut 24, dont les diviseurs (1, 2, 3, 4, 6, 8, 12, 24) constituent les longueurs de clé candidates. En combinant cette information avec le test de l'indice de coïncidence pour chaque diviseur plausible (typiquement entre 3 et 12 pour une clé raisonnable), on retient généralement une valeur unique de L , par exemple $L = 6$ dans cet exemple fictif, avant de passer à l'étape 2.

4. Autres chiffrements classiques

- **Chiffre de Playfair** (digrammes) : chiffre chaque paire de lettres consécutives selon une grille 5×5 construite à partir d'un mot-clé, ce qui masque partiellement les statistiques mono-lettre. Il résiste donc mieux à l'analyse fréquentielle simple, mais reste vulnérable à l'analyse des digrammes fréquents (certaines paires de lettres, comme « ES », « LE », « DE » en français, restent statistiquement dominantes), combinée à des contraintes structurelles propres à Playfair (pas de digramme composé de deux lettres identiques, par exemple), qui réduisent l'espace des hypothèses.
- **Chiffre de transposition** (permutation des lettres sans substitution) : la distribution de fréquence des lettres est conservée à l'identique (contrairement à la substitution), ce qui trahit immédiatement ce type de chiffrement (un simple calcul de fréquence, comparé au profil de la langue, révèle une correspondance quasi parfaite lettre à lettre) et oriente vers une cryptanalyse par recherche d'anagrammes/permutations, souvent guidée par la longueur du texte (indice sur la taille probable de la grille de transposition) et par des mots probables (*cribs*) recherchés sous forme de permutations locales.
- **Chiffres homophoniques** : pour aplatir la distribution de fréquence, certaines variantes historiques associent à une lettre fréquente (comme le E) plusieurs symboles chiffrés différents

utilisés alternativement. Cette technique complique l'analyse fréquentielle simple au niveau des lettres, mais laisse généralement subsister des biais exploitables au niveau des digrammes et de la structure du langage, et reste cassable avec davantage de texte chiffré et de patience.

5. Ce que cette cryptanalyse historique enseigne

- Toute redondance statistique du texte clair qui « survit » au chiffrement est exploitable — principe qui reste valable pour évaluer des schémas modernes mal conçus (ex. mode ECB, chapitre TP3).
- Un espace de clé trop petit (chiffre de César : 25 clés) rend l'attaque exhaustive triviale, quelle que soit la qualité apparente de l'algorithme.
- Un espace de clé très grand (substitution mono-alphabétique : 26!) ne suffit pas à garantir la sécurité si une propriété statistique (ici, la fréquence des lettres) fuit malgré le chiffrement — la taille de l'espace de clé est une condition **nécessaire**, jamais **suffisante**.
- La confusion (rendre la relation clé/texte chiffré complexe) et la diffusion (répartir l'influence de chaque bit du clair sur tout le chiffré), concepts formalisés plus tard par Claude Shannon, sont directement motivées par la nécessité de résister à l'analyse fréquentielle : un bon chiffrement moderne doit, par construction, produire un texte chiffré statistiquement indiscernable d'une suite aléatoire.

Approfondissement : de la cryptanalyse manuelle à la cryptanalyse assistée par ordinateur

Les techniques présentées dans ce chapitre (comptage de fréquences, calcul de l'indice de coïncidence, recherche de répétitions) étaient historiquement réalisées à la main, ce qui limitait la longueur des textes analysables et le nombre d'hypothèses testées. Leur automatisation informatique — même avec des scripts très simples comme ceux présentés ci-dessus — change radicalement l'échelle de l'attaque : un ordinateur personnel actuel peut tester des milliers d'hypothèses de longueur de clé et des millions de sous-hypothèses de décalage en une fraction de seconde. Ce constat annonce un thème central du chapitre 3 : à mesure que la puissance de calcul disponible augmente, la marge de sécurité qu'un concepteur d'algorithme doit intégrer doit croître en conséquence.

À retenir

- L'analyse fréquentielle casse toute substitution mono-alphabétique, quelle que soit la taille apparente de son espace de clé (26! permutations).
- Le chiffre de Vigenère se casse en deux temps : détermination de la longueur de clé (Kasiski, indice de coïncidence, formule $IC = \sum n_i(n_i-1) / N(N-1)$), puis cryptanalyse fréquentielle de chaque sous-alphabet.
- Playfair et les chiffres homophoniques compliquent l'analyse fréquentielle simple mais restent vulnérables à des attaques statistiques d'ordre supérieur (digrammes, contraintes structurelles).
- Ces méthodes historiques posent les bases conceptuelles (confusion, diffusion, exploitation de redondance) de la cryptanalyse moderne, et leur automatisation informatique illustre pourquoi la marge de sécurité des algorithmes doit suivre la croissance de la puissance de calcul disponible.

Chapitre 3 — Cryptanalyse des chiffrements symétriques modernes

1. Attaque par force brute (exhaustive key search)

Pour un chiffrement par bloc à clé de n bits, l'attaque par force brute nécessite en moyenne $2^{(n-1)}$ essais (on s'attend statistiquement à trouver la bonne clé à mi-parcours de l'espace des 2^n clés possibles, en testant chaque hypothèse à l'aide d'un couple clair/chiffré connu). C'est pourquoi la taille de clé est le premier paramètre de sécurité : DES (56 bits) est aujourd'hui cassable par force brute en quelques heures avec du matériel spécialisé, ce qui l'a rendu obsolète au profit d'AES (128, 192 ou 256 bits), hors de portée de la force brute avec la technologie actuelle et prévisible.

Zoom : structure de DES et controverse historique des boîtes-S

DES (*Data Encryption Standard*), standardisé en 1977, repose sur un **réseau de Feistel** à 16 tours : à chaque tour, la moitié droite du bloc est transformée par une fonction dépendant d'une sous-clé de tour puis combinée par XOR à la moitié gauche, les deux moitiés étant ensuite échangées. Le cœur non linéaire de cette fonction repose sur huit **boîtes-S** (S-boxes), tables de substitution fixes qui introduisent la confusion nécessaire à la sécurité.

DES, dérivé de l'algorithme Lucifer d'IBM, a vu ses boîtes-S modifiées par la NSA avant standardisation, ce qui a suscité une controverse durable sur d'éventuelles portes dérobées. L'historique a montré, avec la publication académique de la cryptanalyse différentielle par Biham et Shamir à la fin des années 1980, que ces modifications renforçaient en réalité la résistance de DES à cette attaque — laissant penser que la NSA connaissait déjà cette technique en interne avant sa découverte publique. Cet épisode illustre un principe de gouvernance important en cryptographie : la confiance dans un algorithme se construit par un examen public ouvert, pas par la confiance dans une autorité de conception, aussi compétente soit-elle.

2. Cryptanalyse différentielle

Introduite publiquement par Biham et Shamir (fin des années 1980) contre DES. Principe :

1. Choisir une paire de textes clairs présentant une différence fixe (souvent un XOR précis), notée Δ .
2. Chiffrer les deux textes et observer la différence entre les chiffrés correspondants.
3. Certaines différences de sortie apparaissent avec une probabilité plus élevée que si le chiffrement était parfaitement aléatoire : ce biais statistique permet de déduire de l'information sur des bits de la clé (ou de sous-clés de tour).
4. Répéter sur de nombreuses paires pour affiner la déduction.

DES s'est révélé partiellement vulnérable à cette technique mais sa conception (notamment le choix des boîtes-S) avait déjà anticipé et limité cette attaque — la NSA connaissait apparemment la

cryptanalyse différentielle avant sa publication académique.

Illustration pédagogique simplifiée d'un biais différentiel

Considérons un chiffrement par bloc jouet totalement fictif, à but strictement illustratif, opérant sur des blocs de 4 bits avec une seule boîte-S non linéaire. Une **caractéristique différentielle** décrit la probabilité qu'une différence d'entrée Δ_{in} produise une différence de sortie Δ_{out} donnée, après passage par la boîte-S :

```
# Boîte-S jouet fictive (4 bits -> 4 bits), à but strictement pédagogique
SBOX_JOUEUR = [0xE, 0x4, 0xD, 0x1, 0x2, 0xF, 0xB, 0x8,
               0x3, 0xA, 0x6, 0xC, 0x5, 0x9, 0x0, 0x7]

def table_différentielle(sbox):
    """Construit la table de distribution des différences (DDT) :
    pour chaque différence d'entrée, compte combien de paires
    d'entrées produisent chaque différence de sortie."""
    taille = len(sbox)
    table = [[0] * taille for _ in range(taille)]
    for x in range(taille):
        for delta_in in range(taille):
            x2 = x ^ delta_in
            delta_out = sbox[x] ^ sbox[x2]
            table[delta_in][delta_out] += 1
    return table

# Une caractéristique différentielle "intéressante" pour un cryptanalyste
# est une paire (delta_in, delta_out) dont le comptage dans la table
# dépasse significativement la moyenne attendue par hasard (taille / 2^n).
```

Dans un chiffrement à plusieurs tours, l'attaquant enchaîne des caractéristiques différentielles tour après tour pour construire une **caractéristique différentielle globale** dont la probabilité, bien que faible, reste significativement supérieure à celle attendue pour un chiffrement parfaitement aléatoire ($2^{-(n)}$ pour un bloc de n bits). En chiffrant un grand nombre de paires respectant la différence d'entrée attendue et en comptant les occurrences de la différence de sortie prédite, les sous-clés cohérentes avec cette observation se distinguent statistiquement des sous-clés incorrectes — c'est le principe du **comptage de clés candidates** au dernier tour, répété jusqu'à isoler la bonne sous-clé.

La résistance d'un algorithme à la cryptanalyse différentielle dépend directement de la qualité de ses boîtes-S (une boîte-S "idéale" minimise le plus grand coefficient de sa table de distribution des différences) et du nombre de tours : plus il y a de tours, plus la probabilité de la caractéristique globale s'effondre, jusqu'à devenir inexploitable en pratique.

3. Cryptanalyse linéaire

Introduite par Matsui (1993). Principe : rechercher des approximations linéaires (combinaisons XOR de bits d'entrée, de sortie et de clé) vraies avec une probabilité significativement différente de $1/2$. Une approximation suffisamment biaisée, combinée à un grand nombre de couples clair/chiffré connus, permet de retrouver des bits de clé par vote statistique (lemme du pilage/*piling-up lemma*).

Le lemme du pilage (piling-up lemma)

Si l'on combine k approximations linéaires indépendantes, chacune vraie avec une probabilité $(1/2 + \varepsilon_i)$ où ε_i est un petit biais, la probabilité que la combinaison (XOR) de toutes les approximations soit vraie s'exprime par :

$$P = 1/2 + 2^{-(k-1)} \times \prod \varepsilon_i$$

Ce lemme montre que le biais global décroît très rapidement (de façon multiplicative) lorsque l'on enchaîne plusieurs approximations à travers les tours successifs d'un chiffrement — d'où l'importance, pour un concepteur d'algorithme, de garantir qu'aucune boîte-S ne présente d'approximation linéaire trop biaisée, et de multiplier le nombre de tours pour que le biais cumulé devienne négligeable. Du point de vue de l'attaquant, le nombre de couples clair/chiffré nécessaires pour exploiter un biais ε croît en $1/\varepsilon^2$, ce qui borne directement la faisabilité pratique de l'attaque : un biais minuscule exige un volume de données irréaliste à collecter.

4. Pourquoi AES résiste-t-il mieux ?

AES a été conçu (concours public NIST, choix de Rijndael en 2001) en intégrant explicitement des critères de résistance à la cryptanalyse différentielle et linéaire dès sa conception (structure des boîtes-S optimisée, nombre de tours dimensionné avec marge de sécurité). Aucune attaque académique connue à ce jour ne casse AES pleine puissance plus rapidement qu'une recherche exhaustive proche de l'optimum théorique.

Zoom : structure SPN d'AES

Contrairement à DES (réseau de Feistel), AES est un **réseau de substitution-permutation (SPN)** : chaque tour applique une substitution non linéaire (une unique boîte-S appliquée octet par octet, construite à partir de l'inverse dans un corps fini $GF(2^8)$, choisie précisément pour minimiser les biais différentiels et linéaires), suivie d'étapes de diffusion linéaire (décalage de lignes, mélange de colonnes) qui propagent rapidement l'influence de chaque octet sur l'ensemble du bloc. Le nombre de tours dépend de la taille de clé : 10 tours pour AES-128, 12 pour AES-192, 14 pour AES-256, un choix qui intègre une marge de sécurité au-delà du nombre de tours strictement nécessaire pour contrer les meilleures cryptanalyses connues au moment de la conception. C'est cette combinaison de boîtes-S mathématiquement bien comprises et de diffusion rapide (dite propriété de « diffusion maximale » sur quatre tours) qui explique la résistance durable d'AES.

5. Attaques structurelles sur les modes opératoires

Un algorithme de bloc robuste (AES) peut néanmoins être compromis par un **mode opératoire** mal choisi ou mal utilisé :

Mode	Faiblesse exploitable
ECB (Electronic Codebook)	des blocs de clair identiques produisent des blocs chiffrés identiques → motifs visibles (voir TP3)
CBC sans authentification	vulnérable aux attaques par <i>padding oracle</i> si un serveur révèle si le padding est valide (voir TP3 et chapitre 6)
Réutilisation de nonce/IV (CTR, GCM)	annule la sécurité du chiffrement en flux résultant, permet de retrouver le XOR de deux messages clairs ; pour GCM, la réutilisation de nonce compromet en plus l'intégrité (falsification de messages)
IV prévisible en CBC	si le vecteur d'initialisation est prévisible par l'attaquant (et non aléatoire), certaines attaques par distinction deviennent possibles sur des messages choisis

Ces attaques ne cassent pas l'algorithme de bloc lui-même : elles exploitent un mauvais usage, ce qui souligne l'importance, en cryptographie appliquée, de suivre des schémas éprouvés (AES-GCM, ChaCha20-Poly1305) plutôt que de composer soi-même bloc + mode.

Illustration : pourquoi ECB fuite des motifs

```
# Démonstration pédagogique (ne pas utiliser ECB en production) :
# chiffrer des blocs de clair identiques en ECB produit des blocs
# chiffrés identiques, ce qui trahit la structure du message d'origine
# (ex. zones uniformes d'une image, champs répétés d'un formulaire).

def chiffrement_ecb_jouet(blocs_clairs, chiffrer_bloc, cle):
    """Simulation conceptuelle : chaque bloc est chiffré indépendamment,
    sans dépendance au bloc précédent (contrairement à CBC/CTR)."""
    return [chiffrer_bloc(bloc, cle) for bloc in blocs_clairs]

# Si blocs_clairs contient deux blocs identiques (ex. b0 == b3),
# alors, quelle que soit la robustesse de chiffrer_bloc (même AES),
# les blocs chiffrés correspondants seront eux aussi identiques :
# c'est une fuite structurelle du MODE, pas de l'algorithme de bloc.
```

6. Attaques par compromis temps-mémoire

Les **tables arc-en-ciel (rainbow tables)** illustrent un compromis entre temps de calcul et espace de stockage pour inverser une fonction à sens unique (utilisées historiquement contre des hachages de mots de passe non salés — approfondi au chapitre 4). Le principe général du compromis temps-mémoire, formalisé par Martin Hellman dès 1980, consiste à précalculer une structure de données (chaînes de réduction/hachage successives) qui permet, lors de l'attaque, de retrouver une préimage en un temps très inférieur à une recherche exhaustive brute, au prix d'un espace de stockage important calculé une seule fois puis réutilisable contre toute cible partageant le même espace de clé (par exemple tous les mots de passe non salés hachés avec le même algorithme).

Approfondissement : pourquoi le salage neutralise ce compromis

L'ajout d'un sel aléatoire unique avant hachage (détaillé au chapitre 4) multiplie l'espace effectif à précalculer par le nombre de sels possibles, rendant une table arc-en-ciel générique inexploitable : l'attaquant devrait précalculer une table distincte pour chaque valeur de sel, ce qui annule l'intérêt économique du précalcul. C'est un exemple typique où une contre-mesure très simple (quelques octets aléatoires) neutralise une classe entière d'attaques par compromis temps-mémoire, sans changer l'algorithme de hachage lui-même.

À retenir

- La taille de clé fixe la borne théorique de résistance à la force brute ($2^{(n-1)}$ essais en moyenne) ; DES est aujourd'hui cassable, AES ne l'est pas avec la technologie actuelle.
- Cryptanalyse différentielle (biais de propagation de différences via la table de distribution des différences) et linéaire (biais d'approximations XOR, lemme du pilage) exploitent des biais statistiques internes à la structure de l'algorithme et ont directement influencé la conception des boîtes-S et du nombre de tours d'AES.
- La structure SPN d'AES (boîtes-S issues de $GF(2^8)$, diffusion rapide, marge de tours) explique sa résistance durable, alors que la structure de Feistel de DES, combinée à une clé trop courte, l'a rendu obsolète.
- La grande majorité des compromissions pratiques de chiffrements symétriques modernes viennent d'un mauvais mode opératoire (ECB, IV/nonce réutilisé, CBC non authentifié) ou d'une mauvaise gestion des paramètres, pas d'une faille dans l'algorithme de bloc lui-même.
- Les compromis temps-mémoire (tables arc-en-ciel) illustrent qu'un précalcul ciblé peut rendre une attaque exhaustive plus rapide qu'attendu, mais restent neutralisés par des contre-mesures simples comme le salage.

Chapitre 4 — Attaques sur les fonctions de hachage

1. Rappel des propriétés attendues

Une fonction de hachage cryptographique doit garantir : la **résistance à la préimage** (impossible de retrouver un message à partir de son empreinte), la **résistance à la seconde préimage** (impossible de trouver un autre message ayant la même empreinte qu'un message donné), et la **résistance aux collisions** (impossible de trouver deux messages distincts ayant la même empreinte).

Ces trois propriétés sont hiérarchisées mais pas totalement indépendantes : une fonction qui ne résiste pas aux collisions n'est pas nécessairement vulnérable à la recherche de préimage (retrouver un message précis reste souvent plus coûteux que trouver deux messages arbitraires en collision), ce qui explique que MD5 ou SHA-1, bien que cassés pour les collisions, ne le soient pas au même degré pour la préimage — une nuance importante pour évaluer le risque réel dans un contexte d'usage donné (signature de document vs vérification d'intégrité simple).

2. Le paradoxe des anniversaires et l'attaque par collision

Pour une empreinte de n bits, trouver une collision par force brute « naïve » coûterait en théorie 2^n essais. Le **paradoxe des anniversaires** montre qu'il suffit en réalité d'environ $2^{(n/2)}$ essais pour trouver une collision avec une probabilité significative (par analogie : dans un groupe de 23 personnes, la probabilité que deux partagent un anniversaire dépasse 50 %, bien moins que les 365 attendus intuitivement).

Justification mathématique du paradoxe

Pour un espace de N valeurs possibles ($N = 365$ pour les anniversaires, $N = 2^n$ pour une empreinte de n bits), la probabilité qu'aucune collision n'apparaisse parmi k tirages aléatoires indépendants s'approxime, pour k petit devant N , par :

$$P(\text{pas de collision}) \approx \exp\left(-k^2 / (2N)\right)$$

La probabilité qu'une collision apparaisse dépasse donc 50 % dès que $k \approx 1,177 \times \sqrt{N}$, soit un ordre de grandeur en **racine carrée de N** — d'où le nombre d'essais en $2^{(n/2)}$ pour une empreinte de n bits (puisque $N = 2^n$, $\sqrt{N} = 2^{(n/2)}$). C'est cette croissance en racine carrée, bien plus lente que la croissance linéaire naïve attendue intuitivement, qui rend l'attaque par collision beaucoup plus praticable que l'attaque par préimage (qui reste, elle, en 2^n).

Illustration pédagogique sur un espace réduit

```
import random

def demo_paradoxe_anniversaires(bits_empreinte=16, essais_max=200000):
    """Démonstration pédagogique sur une empreinte volontairement réduite
    (16 bits, soit 65536 valeurs possibles) pour observer en pratique
    la croissance en racine carrée annoncée par la théorie. Ne pas
    confondre avec un hachage cryptographique réel : ceci est un
    tirage aléatoire uniforme simulé, à but strictement illustratif."""
    espace = 2 ** bits_empreinte
    vues = {}
    for tentative in range(1, essais_max + 1):
        empreinte_simulee = random.randrange(espace)
        if empreinte_simulee in vues:
            return tentative # nombre de tirages avant la première collision
        vues[empreinte_simulee] = tentative
    return None # pas de collision trouvée dans la limite fixée

# La théorie prédit une collision attendue autour de  $\sim 1.25 * \sqrt{2^{16}} \approx 320$  tirages,
# très loin des 65536 valeurs de l'espace complet.
```

En exécutant cette simulation plusieurs fois, on observe empiriquement que la première collision survient typiquement après quelques centaines de tirages seulement — pas après des dizaines de milliers — ce qui illustre concrètement pourquoi la sécurité effective d'une fonction de hachage contre les collisions est de $n/2$ bits et non n bits.

Conséquence directe : une fonction de hachage de n bits n'offre qu'une sécurité de $n/2$ bits contre les collisions, ce qui explique pourquoi MD5 (128 bits, soit 64 bits de sécurité en collision — déjà faible en théorie, et en pratique bien pire à cause de faiblesses structurelles additionnelles) est aujourd'hui totalement cassé en pratique, et pourquoi SHA-1 (160 bits, soit 80 bits de sécurité théorique en collision) est déconseillé depuis la démonstration d'une collision pratique en 2017 (attaque « SHAttered »), obtenue bien plus rapidement que 2^{80} grâce à des faiblesses structurelles de l'algorithme, pas seulement grâce au paradoxe des anniversaires générique.

3. Attaques historiques marquantes

- **MD5** : des faiblesses structurelles permettant de construire des collisions bien plus rapidement que la borne générique du paradoxe des anniversaires sont publiées par Xiaoyun Wang et ses coauteurs en 2004-2005 ; des collisions pratiques deviennent alors accessibles avec des moyens de calcul modestes, exploitées ensuite pour forger de faux certificats numériques (des chercheurs ont démontré en 2008 la possibilité de construire un faux certificat d'autorité de certification exploitant une collision MD5 à préfixe choisi, puis l'affaire du malware Flame en 2012 a confirmé l'exploitation réelle d'une technique de collision MD5 pour falsifier une signature de code).
- **SHA-1** : collision pratique démontrée par des chercheurs de Google et du CWI (centre de recherche néerlandais) en 2017 (attaque nommée « SHAttered »), accélérant la migration généralisée vers SHA-2/SHA-3 dans les navigateurs, systèmes de contrôle de version et infrastructures à clé publique.

- **SHA-2 (SHA-256, SHA-512) et SHA-3** : à ce jour, aucune attaque pratique ne remet en cause leur sécurité ; ce sont les standards recommandés. SHA-3, basé sur une construction fondamentalement différente de Merkle-Damgård (la construction en éponge, *sponge construction*, dérivée de l'algorithme Keccak), a été standardisé en partie pour offrir une alternative structurellement indépendante en cas de découverte future d'une faiblesse générique affectant les constructions Merkle-Damgård.

4. Attaque par extension de longueur (length extension attack)

Les fonctions basées sur la construction de Merkle-Damgård (MD5, SHA-1, SHA-2) sont vulnérables à l'attaque par extension de longueur : connaissant $H(\text{message})$ et la longueur de `message` (mais pas `message` lui-même), un attaquant peut calculer $H(\text{message} || \text{padding} || \text{extension})$ pour une extension de son choix, **sans connaître** `message`.

Pourquoi cette attaque fonctionne : la structure interne de Merkle-Damgård

Une fonction de hachage Merkle-Damgård traite le message par blocs successifs : un état interne (accumulateur) est mis à jour bloc par bloc par une fonction de compression, et l'empreinte finale correspond simplement à la valeur de cet état interne après le dernier bloc (padding inclus). Autrement dit, $H(\text{message})$ **n'est rien d'autre que l'état interne complet** de l'algorithme à la fin du traitement — un attaquant qui connaît cette valeur dispose donc de tout ce qu'il faut pour reprendre le calcul là où il s'est arrêté et continuer à traiter des blocs supplémentaires, exactement comme le ferait le détenteur légitime du message original, sans jamais avoir besoin de connaître ce message.

```
# Pseudo-code conceptuel illustrant le principe (pas une implémentation
# réelle d'un algorithme de hachage) :

def hash_merkle_damgard(message, etat_initial, fonction_compression):
    etat = etat_initial
    for bloc in decouper_en_blocs(ajouter_padding(message)):
        etat = fonction_compression(etat, bloc)
    return etat # l'empreinte EST l'état interne final

def attaque_extension_longueur(empreinte_connue, longueur_message_original,
                                extension, fonction_compression):
    """L'attaquant réutilise l'empreinte connue comme état de départ :
    il n'a jamais eu besoin de connaître le message original."""
    etat = empreinte_connue # état interne récupéré tel quel
    # Il faut recréer le padding qu'aurait ajouté l'algorithme au
    # message original (dépend de sa longueur, connue ou devinée),
    # puis reprendre le traitement avec les blocs de l'extension.
    for bloc in decouper_en_blocs(extension):
        etat = fonction_compression(etat, bloc)
    return etat # empreinte valide de (message_original || padding || extension)
```

Conséquence pratique : construire une authentification de message par simple concaténation $H(\text{clé} || \text{message})$ est dangereux — un attaquant peut forger un MAC valide pour un message étendu sans connaître la clé, en repartant directement de l'empreinte publiée. La construction **HMAC** (chapitre Cryptographie) a été spécifiquement conçue pour neutraliser cette attaque, en appliquant la fonction de

hachage **deux fois** avec des clés dérivées différentes ($\text{HMAC}(K, m) = H((K' \oplus \text{opad}) \parallel H((K' \oplus \text{ipad}) \parallel m))$), de sorte que l'attaquant ne dispose jamais directement d'un état interne réutilisable correspondant à un secret.

5. Hachage de mots de passe : attaques et contre-mesures

Attaque	Principe	Contre-mesure
Force brute / dictionnaire	tester des mots de passe candidats et comparer les empreintes	politique de mots de passe robustes, limitation du nombre d'essais
Table arc-en-ciel	tables précalculées d'empreintes pour inverser rapidement un hachage non salé	salage (valeur aléatoire unique par mot de passe, ajoutée avant hachage)
Calcul massivement parallèle (GPU/ASIC)	les fonctions de hachage génériques (SHA-256) sont très rapides à calculer, donc favorables à l'attaquant	fonctions spécialement conçues pour être lentes : bcrypt, scrypt, Argon2 (facteur de coût réglable)

Approfondissement : pourquoi la lenteur volontaire est une propriété de sécurité

Une fonction de hachage générique comme SHA-256 est conçue pour être **rapide**, une propriété désirable pour vérifier l'intégrité d'un fichier volumineux, mais désastreuse pour le hachage de mots de passe : un attaquant disposant de matériel parallèle (GPU, voire circuits dédiés ASIC) peut tester un très grand nombre de candidats par seconde. Les fonctions dédiées au hachage de mots de passe introduisent délibérément un coût :

- **bcrypt** : coût réglable par un facteur de travail exponentiel (chaque incrément double le temps de calcul), basé sur une variante de l'algorithme de chiffrement Blowfish.
- **scrypt** : ajoute une exigence de **mémoire** importante et réglable, ce qui pénalise spécifiquement les architectures matérielles massivement parallèles (GPU/ASIC), moins efficaces lorsqu'elles doivent aussi gérer beaucoup de mémoire par unité de calcul.
- **Argon2** (vainqueur de la compétition *Password Hashing Competition*, recommandé aujourd'hui par défaut) : généralise ce principe en un algorithme **résistant à la mémoire** (*memory-hard*) avec trois paramètres réglables indépendamment — coût en temps, coût en mémoire, et degré de parallélisme — permettant d'ajuster finement le compromis sécurité/performance selon le contexte de déploiement.

Le point commun de ces trois familles est de rendre le coût unitaire d'un essai de mot de passe suffisamment élevé pour qu'un attaquant, même équipé de matériel puissant, ne puisse tester qu'un nombre limité de candidats dans un temps raisonnable — contrairement à une fonction générique rapide, où le facteur limitant est presque uniquement la puissance de calcul brute disponible.

6. Méthodologie d'évaluation d'une fonction de hachage

1. Vérifier la taille de sortie (borne théorique de sécurité en collision = taille/2).

2. Vérifier l'absence d'attaques publiées de préimage/collision pratiques, et distinguer une faiblesse purement théorique (réduction de complexité en dessous de la borne générique, sans attaque praticable) d'une attaque réellement exploitable avec des ressources réalistes.
3. Vérifier le contexte d'usage : une fonction sûre pour l'intégrité de fichiers (SHA-256) ne l'est pas nécessairement pour le hachage de mots de passe sans adaptation (facteur de travail, salage) — un hachage rapide et une fonction dédiée lente répondent à des menaces différentes.
4. Vérifier, pour tout usage impliquant un secret (MAC), l'emploi d'une construction dédiée (HMAC) plutôt qu'une concaténation naïve $H(\text{clé} || \text{message})$, vulnérable à l'extension de longueur pour les constructions Merkle-Damgård.

À retenir

- Le paradoxe des anniversaires ($P(\text{collision}) \approx 1 - \exp(-k^2/2N)$) divise par deux, en bits, la sécurité effective d'une fonction de hachage contre les collisions : n bits de sortie n'offrent que $n/2$ bits de sécurité en collision.
- MD5 et SHA-1 sont cassés en pratique pour les collisions (bien plus rapidement que la borne générique du paradoxe des anniversaires, grâce à des faiblesses structurelles) et ne doivent plus être utilisés pour un usage sécuritaire ; SHA-2 et SHA-3 restent recommandés.
- L'attaque par extension de longueur exploite le fait que l'empreinte d'une fonction Merkle-Damgård EST son état interne final, réutilisable directement par un attaquant : c'est pourquoi HMAC, et non une simple concaténation $H(\text{clé} || \text{message})$, doit être utilisé pour authentifier des messages.
- Le hachage de mots de passe exige des fonctions dédiées (bcrypt, scrypt, Argon2), volontairement lentes, coûteuses en mémoire et salées — jamais une fonction générique seule (SHA-256 brut), rapide par conception et donc favorable à un attaquant équipé de matériel parallèle.

Chapitre 5 — Cryptanalyse des systèmes asymétriques

1. RSA : rappel du principe et hypothèse de sécurité

La sécurité de RSA repose sur la difficulté supposée de **factoriser un grand nombre entier** produit de deux grands nombres premiers ($n = p \times q$). Aucun algorithme classique connu ne factorise efficacement (en temps polynomial) un nombre suffisamment grand — c'est une hypothèse calculatoire, pas une preuve mathématique absolue.

Rappel des relations mathématiques de RSA

Pour construire une paire de clés, on choisit deux grands nombres premiers distincts p et q , on calcule $n = p \times q$ (le module, rendu public) et l'indicatrice d'Euler $\varphi(n) = (p-1)(q-1)$. On choisit un exposant public e premier avec $\varphi(n)$ (souvent 65537 en pratique, un choix qui offre un bon compromis entre rapidité de chiffrement et sécurité), puis on calcule l'exposant privé d , l'inverse modulaire de e modulo $\varphi(n)$: $e \times d \equiv 1 \pmod{\varphi(n)}$. Le chiffrement d'un message m s'écrit $c = m^e \bmod n$, et le déchiffrement $m = c^d \bmod n$.

La sécurité repose entièrement sur le fait que, connaissant uniquement n et e (les valeurs publiques), un attaquant ne peut pas calculer d efficacement — ce qui, à ce jour, exige de connaître $\varphi(n)$, ce qui exige à son tour de connaître p et q , c'est-à-dire de **factoriser n** . C'est cette chaîne de réductions qui relie directement la sécurité pratique de RSA au problème mathématique de la factorisation.

Exemple pédagogique à très petits nombres (à but strictement illustratif — jamais utilisable en pratique)

L'exemple suivant, avec des nombres volontairement minuscules pour rester calculable à la main, illustre le mécanisme (il ne représente évidemment aucune sécurité réelle) :

```

# Exemple RSA pédagogique à très petits nombres – usage strictement
# didactique, à des lieues de toute sécurité réelle (des clés RSA
# réelles utilisent des nombres premiers de centaines de chiffres).

p, q = 61, 53
n = p * q                # n = 3233 (module public)
phi = (p - 1) * (q - 1)  # phi(n) = 3120
e = 17                   # exposant public choisi, premier avec phi(n)

def inverse_modulaire(e, phi):
    # Algorithme d'Euclide étendu (version simplifiée à but pédagogique)
    for d in range(2, phi):
        if (e * d) % phi == 1:
            return d
    return None

d = inverse_modulaire(e, phi)  # d = 2753 (exposant privé)

m = 65                    # message clair jouet (un petit entier)
c = pow(m, e, n)          # chiffrement : c = m^e mod n
m_retrouve = pow(c, d, n)  # déchiffrement : m = c^d mod n
assert m_retrouve == m

```

Un attaquant ne connaissant que $n = 3233$ et $e = 17$ devrait, pour retrouver d , factoriser n en $p = 61$ et $q = 53$ — trivial ici par essai de division, mais totalement impraticable dès que p et q comptent plusieurs centaines de chiffres, ce qui est le cas des clés RSA réelles (2048 bits ou plus, soit des nombres premiers d'environ 300 chiffres décimaux chacun).

2. Attaques par factorisation

- **Force brute / essai de division** : totalement impraticable pour des clés de taille réaliste (2048 bits ou plus).
- **Méthode de Fermat** : exploite le fait que si p et q sont proches l'un de l'autre, n peut s'écrire comme une différence de deux carrés ($n = a^2 - b^2$), ce qui permet de retrouver rapidement les facteurs en cherchant a tel que $a^2 - n$ soit un carré parfait. Cette méthode illustre pourquoi les générateurs de clés RSA doivent s'assurer que p et q sont suffisamment éloignés l'un de l'autre.
- **Algorithmes sous-exponentiels** : le *General Number Field Sieve* (GNFS) est l'algorithme classique le plus efficace connu pour factoriser de grands nombres ; sa complexité, bien que sous-exponentielle (plus rapide que la force brute, mais bien plus lente qu'un algorithme polynomial), reste hors de portée pour des clés RSA de 2048 bits avec la technologie actuelle. C'est pourquoi la taille de clé recommandée a augmenté avec le temps (512 bits cassé dès 1999, 768 bits cassé en 2009, 1024 bits considéré fragilisé, 2048 bits recommandé aujourd'hui).
- **Algorithme rho de Pollard** : une méthode probabiliste, plus efficace que l'essai de division mais toujours de complexité exponentielle dans le nombre de bits, souvent utilisée pour trouver rapidement de petits facteurs avant de recourir à une méthode plus lourde comme le GNFS.
- **Menace quantique** : l'algorithme de Shor, exécuté sur un ordinateur quantique suffisamment puissant, factoriserait un grand nombre en temps polynomial — casserait RSA (et le logarithme discret, donc Diffie-Hellman/ECC classiques). Cette menace motive la standardisation en cours d'algorithmes post-quantiques (ex. NIST PQC : ML-KEM/Kyber, ML-DSA/Dilithium).

3. Attaques exploitant une mauvaise implémentation de RSA

Ces attaques ne cassent pas le problème mathématique sous-jacent mais exploitent des erreurs pratiques, bien plus fréquentes en réalité :

Attaque	Condition d'application
Exposant public e trop petit sans padding ($e = 3$)	message court, chiffré sans padding correct (OAEP), permet une racine cubique directe
Modules partagés / clés faibles	deux clés RSA différentes partagent accidentellement un facteur premier → PGCD des deux modules révèle instantanément la factorisation
Attaque de Wiener	exposant privé d choisi trop petit par rapport au module → récupération de d par développement en fractions continues
Attaque de Coppersmith	conditions particulières sur des bits connus de p , q ou du message (par exemple une fraction significative des bits de p déjà connue)
Attaque de diffusion (Håstad)	un même message chiffré avec le même petit exposant public ($e = 3$) vers plusieurs destinataires disposant de modules différents → combinaison des chiffrés (théorème des restes chinois) pour retrouver le message sans factoriser aucun module
Réutilisation de nonce / mauvaise génération aléatoire	générateur pseudo-aléatoire faible lors de la génération des nombres premiers → cas réel documenté sur des équipements embarqués bon marché

Approfondissement : intuition de l'attaque du module commun / PGCD

Si deux modules RSA distincts $n_1 = p \times q_1$ et $n_2 = p \times q_2$ partagent accidentellement le même facteur premier p (par exemple à cause d'un générateur de nombres aléatoires de mauvaise qualité produisant des premiers avec trop peu d'entropie), alors le calcul du plus grand commun diviseur $\text{PGCD}(n_1, n_2)$ révèle instantanément p , sans aucune tentative de factorisation classique — un simple algorithme d'Euclide, de complexité négligeable, suffit. C'est une illustration frappante de la section 1 du chapitre 1 : un problème mathématiquement difficile (factoriser n) peut devenir trivial dès qu'une hypothèse implicite de génération (indépendance et aléa suffisant des nombres premiers) est violée en pratique.

Approfondissement : intuition de l'attaque de Wiener

Si l'exposant privé d est choisi trop petit (approximativement $d < n^{0,25}$, une borne précisée par les travaux ultérieurs de Boneh et Durfee qui l'ont étendue), la fraction e/n admet un développement en fractions continues dont l'un des convergents permet de retrouver d directement, en temps polynomial. Cette attaque illustre pourquoi un exposant privé « pratique » (choisi petit pour accélérer le déchiffrement) est une optimisation dangereuse : la norme actuelle recommande un exposant privé de taille comparable au module lui-même, quitte à accepter un coût de déchiffrement plus élevé (souvent compensé en pratique par le théorème des restes chinois, qui accélère le calcul sans réduire d).

4. Diffie-Hellman et logarithme discret

La sécurité de Diffie-Hellman (et d'ElGamal) repose sur la difficulté du **problème du logarithme discret** : étant donné g , p et $g^x \bmod p$, retrouver x est supposé difficile. Les meilleures attaques classiques connues (*index calculus*) ont une complexité comparable à la factorisation RSA pour une taille de groupe équivalente, d'où des recommandations de taille de clé similaires.

L'algorithme d'*index calculus* exploite une structure particulière des groupes multiplicatifs modulo un nombre premier (contrairement aux courbes elliptiques, voir section 5) : il précalcule les logarithmes discrets d'un ensemble de « petits » nombres premiers (une base de facteurs), puis exprime le logarithme cible comme combinaison de ces valeurs précalculées. C'est cette structure exploitable qui explique pourquoi le logarithme discret sur les entiers modulo un nombre premier nécessite des tailles de clé comparables à celles de RSA (2048 bits ou plus), contrairement au logarithme discret sur courbe elliptique, dépourvu de structure équivalente exploitable par *index calculus*.

Attaque pratique notable : l'attaque *Logjam* (2015) a montré que de nombreux serveurs réutilisaient les mêmes petits groupes premiers standardisés pour Diffie-Hellman, permettant un précalcul mutualisé (la partie la plus coûteuse d'*index calculus* ne dépend que du groupe, pas de la clé spécifique) rendant l'attaque de logarithme discret praticable contre des groupes de 512 bits — illustration qu'un paramètre partagé et sous-dimensionné affaiblit tous les systèmes qui le réutilisent, un précalcul unique amortissant son coût sur un très grand nombre de cibles.

5. Courbes elliptiques (ECC)

Le problème du logarithme discret sur courbe elliptique (ECDLP) est considéré plus difficile, à taille de clé égale, que son équivalent sur les entiers : une clé ECC de 256 bits offre un niveau de sécurité comparable à une clé RSA de 3072 bits, pour un coût de calcul et une taille de clé bien moindres — ce qui explique l'adoption croissante d'ECC (ECDSA, X25519) dans les protocoles modernes (TLS 1.3, SSH).

Pourquoi ECDLP résiste-t-il mieux, à taille égale ?

Le groupe des points d'une courbe elliptique ne possède pas de structure exploitable équivalente à celle utilisée par *index calculus* sur les entiers : les meilleures attaques génériques connues contre ECDLP, comme l'**algorithme rho de Pollard adapté aux courbes elliptiques**, ont une complexité en $O(\sqrt{N})$, où N est l'ordre du groupe — c'est-à-dire une complexité purement exponentielle, comparable à une recherche exhaustive optimisée par le paradoxe des anniversaires (voir chapitre 4), sans raccourci structurel connu. C'est cette absence de structure exploitable qui permet à ECC d'offrir un niveau de sécurité donné avec des clés bien plus courtes que RSA ou Diffie-Hellman classique.

Point de vigilance : le choix de la courbe

Toutes les courbes elliptiques ne se valent pas : certains paramétrages standardisés ont fait l'objet de suspicions (génération opaque de paramètres, sans justification vérifiable, dans le cas de certaines courbes historiques normalisées par des organismes gouvernementaux), ce qui a motivé l'adoption croissante de courbes dont les paramètres sont générés de façon vérifiable et reproductible (comme

Curve25519), un critère de confiance qui s'ajoute à la seule robustesse mathématique du problème ECDLP.

6. Démarche générale de cryptanalyse d'un système asymétrique

1. Vérifier les paramètres utilisés (taille de clé, courbe/groupe standardisé, largement audité, et non « maison »).
2. Rechercher des erreurs de génération de clé (facteurs partagés, aléa faible, exposant privé anormalement petit).
3. Vérifier l'usage correct du padding (OAEP pour le chiffrement RSA, PSS pour la signature) — un module et un exposant robustes ne protègent en rien contre une utilisation sans padding adapté.
4. Ne considérer la factorisation/logarithme discret « brut » que comme dernier recours théorique : en pratique, l'immense majorité des compromissions viennent des points 1 à 3.

À retenir

- La sécurité de RSA et Diffie-Hellman repose sur des hypothèses calculatoires (factorisation, logarithme discret), pas sur une preuve d'impossibilité absolue ; les meilleurs algorithmes classiques connus (GNFS, index calculus) restent sous-exponentiels mais impraticables aux tailles de clé recommandées actuelles.
- Les attaques réellement exploitées en pratique ciblent presque toujours une mauvaise implémentation (aléa faible menant à des facteurs premiers partagés, exposant privé trop petit exploitable par l'attaque de Wiener, absence de padding OAEP/PSS, diffusion d'un même message à petit exposant) plutôt que l'algorithme mathématique lui-même.
- ECC atteint un niveau de sécurité équivalent à RSA avec des clés bien plus courtes, car le logarithme discret sur courbe elliptique ne possède pas d'équivalent à l'attaque par index calculus qui affaiblit RSA et Diffie-Hellman classique.
- La menace de l'informatique quantique (algorithme de Shor, complexité polynomiale) casserait à la fois RSA, Diffie-Hellman et ECC classiques, ce qui motive la transition en cours vers la cryptographie post-quantique (NIST PQC).

Chapitre 6 — Attaques par canaux auxiliaires et attaques pratiques

1. Principe des attaques par canaux auxiliaires (side-channel)

Une attaque par canal auxiliaire n'exploite pas de faiblesse mathématique de l'algorithme, mais une **information physique fuitée par son implémentation** : temps d'exécution, consommation électrique, rayonnement électromagnétique, comportement du cache processeur, messages d'erreur. Ces attaques rappellent que la sécurité d'un système cryptographique dépend autant de son implémentation que de sa conception mathématique.

On distingue généralement deux grandes familles d'attaques par canal auxiliaire :

- les attaques **passives**, où l'attaquant se contente d'observer une fuite (temps, consommation, rayonnement) sans interagir autrement avec le système ;
- les attaques **actives**, où l'attaquant perturbe volontairement le fonctionnement du dispositif (variation de tension, d'horloge, impulsion laser) pour provoquer une erreur de calcul exploitable — c'est le principe des **attaques par injection de faute** (voir section 7).

2. Attaques temporelles (timing attacks)

Si le temps d'exécution d'une opération cryptographique dépend des bits secrets manipulés (ex. une comparaison de mot de passe qui s'arrête au premier octet différent), un attaquant mesurant précisément ce temps peut déduire de l'information sur le secret, octet par octet.

Contre-mesure : comparaisons en temps constant (*constant-time comparison*), qui parcourent systématiquement toute la donnée indépendamment du résultat intermédiaire.

Illustration : comparaison vulnérable contre comparaison en temps constant

```

# Comparaison VULNÉRABLE : s'arrête dès le premier octet différent,
# donc le temps d'exécution dépend du nombre d'octets corrects trouvés
# avant la première différence -> fuite exploitable par mesure de temps.
def comparaison_vulnerable(secret, tentative):
    if len(secret) != len(tentative):
        return False
    for a, b in zip(secret, tentative):
        if a != b:
            return False    # sortie anticipée : fuite temporelle
    return True

# Comparaison en TEMPS CONSTANT : parcourt systématiquement toute la
# donnée, accumule les différences par XOR, et ne conclut qu'à la fin.
def comparaison_temps_constant(secret, tentative):
    if len(secret) != len(tentative):
        return False
    difference_accumulee = 0
    for a, b in zip(secret, tentative):
        difference_accumulee |= a ^ b    # pas de branchement dépendant du secret
    return difference_accumulee == 0

```

En pratique, il ne suffit pas d'écrire un tel code : le compilateur, le processeur (prédiction de branchement, exécution spéculative) et le cache peuvent réintroduire des variations de temps subtiles. C'est pourquoi les bibliothèques cryptographiques sérieuses fournissent des primitives de comparaison en temps constant testées et auditées, plutôt que de laisser chaque développeur réimplémenter la sienne.

3. Attaques par oracle de padding (padding oracle)

Décrites en détail en TP3 : si un système révèle (même indirectement, par un code d'erreur différent ou un temps de réponse différent) si un texte chiffré déchiffré présente un padding valide selon un schéma comme PKCS#7 en mode CBC, un attaquant peut déchiffrer progressivement l'intégralité du message sans connaître la clé, en interrogeant le système bloc par bloc, octet par octet. Cette classe d'attaque (Vaudenay, 2002) a compromis en pratique de nombreuses implémentations TLS/SSL (ex. attaque *POODLE*, 2014).

Principe algorithmique simplifié de l'attaque

L'attaque exploite la structure du mode CBC : le déchiffrement d'un bloc C_i dépend du bloc chiffré précédent C_{i-1} par un XOR final ($\text{clair}_i = \text{déchiffrement_bloc}(C_i) \oplus C_{i-1}$). En modifiant délibérément des octets du bloc précédent (que l'attaquant contrôle entièrement s'il forge son propre texte chiffré, ou dont il connaît la valeur dans un texte chiffré intercepté) et en observant si le serveur signale un padding valide ou invalide, l'attaquant peut déduire, octet par octet en partant de la fin du bloc, la valeur du clair intermédiaire :

1. Modifier le dernier octet du bloc précédent et soumettre le texte chiffré modifié au système.
2. Observer la réponse : padding valide (le dernier octet du clair déchiffré vaut alors `0x01`, un padding PKCS#7 minimal) ou invalide.

3. En essayant les 256 valeurs possibles de l'octet modifié, retrouver la valeur exacte de l'octet de clair intermédiaire correspondant, par un simple calcul de XOR.
4. Répéter en remontant octet par octet vers le début du bloc, en ajustant les octets déjà retrouvés pour maintenir un padding cohérent (`0x02 0x02`, puis `0x03 0x03 0x03`, etc.), jusqu'à déchiffrer le bloc entier.
5. Répéter l'opération pour chaque bloc du message.

Le coût de cette attaque est de l'ordre de 256 requêtes par octet dans le pire cas, soit un nombre de requêtes linéaire en la taille du message — largement praticable dès lors que l'oracle (le système qui distingue padding valide/invalid) est accessible et interrogeable un grand nombre de fois, ce qui explique la gravité de cette classe d'attaque une fois découverte dans un système réel.

4. Attaques par analyse de consommation électrique et électromagnétique

- **SPA (Simple Power Analysis)** : observation directe de la trace de consommation, révélant parfois la séquence d'opérations effectuées (ex. distinguer un carré d'une multiplication en exponentiation modulaire — une différence de motif visible directement sur une seule trace, sans traitement statistique).
- **DPA (Differential Power Analysis)** : analyse statistique de nombreuses traces de consommation pour un grand nombre d'entrées, permettant de retrouver des bits de clé même en présence de bruit. Le principe consiste à formuler une hypothèse sur une portion de la clé, à prédire la consommation attendue sous cette hypothèse pour chaque trace collectée, puis à corréliser statistiquement cette prédiction à la consommation réellement mesurée : l'hypothèse correcte se distingue des hypothèses incorrectes par une corrélation significativement plus élevée, même lorsque le bruit de mesure rendrait toute lecture directe d'une trace individuelle impossible.
- **CPA (Correlation Power Analysis)** : raffinement statistique de la DPA, utilisant un coefficient de corrélation (par exemple celui de Pearson) plutôt qu'une simple différence de moyennes, ce qui améliore l'efficacité de l'attaque avec moins de traces nécessaires.

Ces attaques concernent principalement des dispositifs physiques (cartes à puce, modules matériels de sécurité, objets connectés) où l'attaquant a un accès physique ou de proximité.

Contre-mesures classiques contre l'analyse de consommation

- **Masquage (masking)** : décomposer chaque valeur secrète manipulée en plusieurs parts aléatoires dont la combinaison reconstitue la valeur réelle, de sorte qu'aucune part individuelle ne corrèle avec le secret.
- **Ajout de bruit matériel** : introduire des variations aléatoires de consommation ou d'horloge pour dégrader le rapport signal/bruit exploité par l'attaquant.
- **Équilibrage des opérations** : concevoir les circuits pour que les opérations sur un bit à 0 et un bit à 1 consomment une énergie similaire, réduisant la fuite exploitable à la source.

5. Attaques par cache (cache-timing attacks)

Exploitent les différences de temps d'accès entre données présentes ou absentes du cache processeur. Des implémentations logicielles d'AES utilisant des tables précalculées (T-tables) indexées par des bits de la clé ont ainsi été attaquées avec succès, motivant l'adoption d'instructions matérielles dédiées (AES-NI) exécutant le chiffrement en temps constant indépendamment de la clé.

Deux techniques classiques d'attaque par cache

- **Prime+Probe** : l'attaquant remplit (« prime ») certaines lignes de cache avec ses propres données, laisse la victime exécuter son calcul, puis mesure (« probe ») le temps d'accès à ses propres données pour déterminer si la victime a évincé certaines lignes de cache — ce qui révèle indirectement quelles zones mémoire (et donc potentiellement quels index de table dépendant de la clé) la victime a consultées.
- **Flush+Reload** : applicable lorsque l'attaquant partage de la mémoire physique avec la victime (bibliothèque partagée, environnement virtualisé partagé), cette technique consiste à vider une ligne de cache précise, laisser la victime s'exécuter, puis mesurer le temps d'accès à cette même ligne pour savoir si la victime l'a rechargée — une technique plus précise que Prime+Probe mais qui exige des conditions de partage mémoire spécifiques.

6. Attaques sur les générateurs de nombres aléatoires

Un générateur de nombres pseudo-aléatoires (PRNG) de mauvaise qualité ou mal initialisé (faible entropie au démarrage, graine prévisible) compromet directement toute construction cryptographique qui en dépend (génération de clés, de nonces, de vecteurs d'initialisation). Cas réel documenté : une faille dans le générateur aléatoire d'une distribution Linux (Debian, 2006-2008) a réduit l'espace des clés SSH générées à quelques dizaines de milliers de valeurs possibles, les rendant énumérables.

Ce que cet exemple enseigne sur la conception d'un générateur cryptographique

Cet incident, provoqué par une modification apparemment mineure du code source affectant l'accumulation d'entropie, illustre plusieurs principes durables :

- toute source d'aléa utilisée en cryptographie doit être clairement identifiée, testée et si possible standardisée (`/dev/urandom` ou un CSPRNG dédié fourni par une bibliothèque audité), plutôt que réimplémentée ;
- une modification apparemment inoffensive d'un composant cryptographique (ici, du point de vue du code source, un changement limité) peut avoir des conséquences catastrophiques et rester non détectée pendant une longue période, ce qui plaide pour des revues de code spécifiquement dédiées aux composants de sécurité, et pour des tests statistiques réguliers de la qualité de l'aléa produit ;
- les conséquences d'un aléa faible sont souvent silencieuses : le système continue de fonctionner normalement, ce qui rend ce type de faille particulièrement difficile à détecter sans audit spécifique.

7. Attaques par injection de faute (fault injection)

Contrairement aux attaques passives précédentes, l'injection de faute est une attaque **active** : l'attaquant perturbe volontairement l'environnement d'exécution (variation de tension d'alimentation, glitch d'horloge, exposition à un rayonnement ciblé) pour provoquer une erreur de calcul à un instant précis de l'exécution d'un algorithme cryptographique. L'analyse de la différence entre un résultat correct et un résultat fauté (**analyse différentielle de faute**, *Differential Fault Analysis*, DFA) peut alors, dans certains schémas, permettre de retrouver la clé secrète bien plus rapidement qu'une attaque purement mathématique — un exemple classique et bien documenté dans la littérature académique concerne les implémentations de RSA utilisant le théorème des restes chinois pour accélérer le calcul, où une seule faute correctement placée peut suffire à révéler un facteur du module.

Contre-mesures contre l'injection de faute

- **Vérification systématique du résultat** avant de le renvoyer (par exemple, pour une signature, vérifier immédiatement sa validité avec la clé publique correspondante avant de la transmettre).
- **Détecteurs matériels** de conditions anormales (tension, horloge, température) déclenchant une réinitialisation ou un blocage du dispositif.
- **Redondance de calcul** (répéter l'opération et comparer les résultats) pour détecter une incohérence provoquée par une perturbation ponctuelle.

8. Démarche défensive face aux canaux auxiliaires

- Utiliser des implémentations cryptographiques éprouvées et auditées (bibliothèques reconnues) plutôt que réimplémenter soi-même des primitives sensibles.
- Privilégier les implémentations en temps constant pour toute opération manipulant un secret, en gardant à l'esprit que le compilateur et le matériel peuvent réintroduire des variations que le code source seul ne laisse pas voir.
- S'assurer d'une source d'entropie fiable pour toute génération aléatoire cryptographique (`/dev/urandom`, CSPRNG dédié), et ne jamais réimplémenter un générateur pseudo-aléatoire cryptographique « maison ».
- Pour les dispositifs physiques sensibles, envisager des contre-mesures matérielles (masquage, bruit ajouté, détecteurs de perturbation) en complément des contre-mesures logicielles.
- Vérifier systématiquement le résultat d'une opération cryptographique sensible avant de le divulguer, comme filet de sécurité contre une éventuelle injection de faute.

À retenir

- Les attaques par canaux auxiliaires exploitent l'implémentation, pas l'algorithme : elles rappellent qu'une preuve mathématique de sécurité ne suffit pas, et se répartissent entre attaques passives (mesure d'une fuite) et attaques actives (injection de faute).
- Le padding oracle est une attaque pratique majeure contre les usages naïfs de CBC (coût de l'ordre de 256 requêtes par octet), à l'origine de plusieurs failles réelles dans TLS/SSL.
- SPA, DPA et CPA exploitent la consommation électrique, avec des contre-mesures dédiées (masquage, bruit, équilibrage) ; Prime+Probe et Flush+Reload exploitent le comportement du cache processeur.

- Un générateur aléatoire de mauvaise qualité peut annuler la sécurité de tout système cryptographique qui en dépend, quelle que soit la robustesse théorique de l'algorithme utilisé — et ce type de défaillance reste souvent silencieux et difficile à détecter sans audit dédié.

Dr. Zakaria Sawadogo — École Polytechnique de Ouagadougou